Corpus Cleanup – CoViBa

Davo Acevedo-Cardona

2026-03-28

Contents

Setup and Introduction: 1
Configuration 2
Step 1 – Remove NOTE blocks and UUID artifacts 2
Step 2 – Normalization of Transcripts 3
Step 3 – Timestamp reconstruction and reassignment: 4
Step 4 – Export Cleaned Transcripts 6

Setup and Introduction:

Automated Transcript Cleanup

Removing Distracting Lines + Correcting format and timestamps

Help: https://stringr.tidyverse.org/articles/regular-expressions.html

This notebook cleans WEBVTT-style `.txt` transcripts: - removes NOTE blocks / junk IDs - normalizes speaker tags - standardizes timestamps and repairs missing ones - writes `Cleaned_*.txt` outputs

This project has been funded in part by National Endowment for the Humanities: Exploring the Human Endeavor. http://www.neh.gov

```r
#Required libraries
#install.packages("readr")
#install.packages("stringr")
#install.packages("chron")
#install.packages("magrittr")

#Load libraries
library(magrittr)
library(readr)
library(stringr)
library(chron)

knitr::opts_chunk$set(
  echo = TRUE,
  message = FALSE,
  warning = FALSE
)
```

Configuration

This step sets the working directory to the `Cleanup` folder and reads all `.txt` transcript files into a data frame for further processing. Edit the paths with the directory where all the data is.

```
# (!) YOUR working directory, ending with "Cleanup" folder (wd):
your_wd <- "C:/Users/ateni/Desktop/U of A/EXPERIMENTOS/CoVIBA"

stopifnot(dir.exists(your_wd))
setwd(your_wd)
getwd()  # confirm correct wd
```

```
## [1] "C:/Users/ateni/Desktop/U of A/EXPERIMENTOS/CoVIBA"
```

```
# Getting file name list from working directory
filelist <- list.files(pattern = "\\.txt$")
stopifnot(length(filelist) > 0)

# Pre-allocate data frame (avoid growing it inside the loop)
files <- data.frame(
  Name = character(length(filelist)),
  Text = character(length(filelist)),
  stringsAsFactors = FALSE
)

# Collect names and text content
for (i in seq_along(filelist)) {
  files$Name[i] <- paste0("Cleaned_", filelist[i])
  files$Text[i] <- read_file(filelist[i])
}
```

Step 1 – Remove NOTE blocks and UUID artifacts

This step removes non-linguistic metadata (NOTE blocks and system-generated UUID strings) from each transcript.

```
transcribe_1 <- function(x) {
  x %>%
    str_remove_all("\r\n\r\nNOTE .*") %>%  #removes NOTE blocks + following blank lines
    str_remove_all("\\w{8}-\\w{4}-\\w{4}-\\w{4}-\\w{12}(-\\w{1})?\\r\\n") # removes UUID-like lines (up
}
```

```
files$Text <- vapply(files$Text, transcribe_1, FUN.VALUE = character(1)) #vapply instead of sapply
```

```
files$end_time <- vapply(files$Text, function(txt) {
  m <- str_extract_all(txt, "--> (\\d{1,2}:\\d{2}:\\d{2}\\.\\d{3})", simplify = TRUE)
  str_replace(m[ncol(m)], "--> ", "")
}, FUN.VALUE = character(1))
```

Step 2 – Normalization of Transcripts

This step performs structural normalization of the cleaned transcripts. While Step 1 removed non-linguistic metadata, this stage standardizes speaker labels, timestamps, spacing, and formatting inconsistencies introduced during transcription or export.

```
transcribe_2 <- function(x) {

  # Regex fragments
  TS  <- "(\\d{1,2}:\\d{2}:\\d{2}\\.\\d{3})"
  TSP <- paste0(TS, " --> ", TS)
  SPK <- "(INV|PAR|INT)" #INT OR INV??????

  x %>%
    # 1) Normalize speaker tags / IDs
    str_replace_all("<\\.?\\s*(INV|PAR|INV)>", "<v \\1>") %>% # original kept "INV|PAR|INV"
    str_replace_all(paste0("(", "<v (INV|PAR|INV)>", ")\\s?(", TSP, ")\\r\\n(.*)"),
                    "\\3\r\n\\1 \\4") %>% # speaker before timestamp
    str_replace_all(paste0("(", SPK, "):"), "<v \\1>") %>% # INV: -> <v INV>
    str_replace_all(paste0("/\\s+<v ", SPK, ">"), "/") %>% # remove speaker IDs after /text/

    # 2) Timestamp
    str_replace_all("-->", "-->") %>%
    str_replace_all("-->", "-->") %>%
    str_replace_all(paste0("(\\d{1,2}:\\d{2}:\\d{2}\\.)(\\d{2,3})"), "\\1000") %>%  # force .000
    str_replace_all("0{3,}:", "00:") %>%

    # 3) Line break / spacing fixes around timestamps
    str_replace_all(paste0("\\n\\n", TS), "\r\n\r\n\\1") %>%
    str_replace_all(paste0("\\s{1,}(", TS, ") -->"), "\r\n\r\n\\1 -->") %>%
    str_replace_all("(\\r\\n\\r\\n)\\s{1}(\\d{1,2})", "\\1\\2") %>%
    str_replace_all("(\\d{3}\\r\\n)\\s{1}", "\\1") %>%
    str_replace_all(".000\\s*\\n<v (INV|PAR|INT)>", ".000\r\n<v \\1>") %>%

    # 4) Collapse timestamps that appear inside a speaker turn
    str_replace_all(
      paste0("\\r\\n\\r\\n\\r?\\n?(", TS, ") --> (", TS, ")\\r?\\n(?!<)"),
      " "
    ) %>%

    # 5) Remove unnecessary line breaks within dialogue
    str_replace_all("([:alnum:]|[:punct:])\\s?\\r\\n([:alnum:]|[:punct:])", "\\1 \\2") %>%
    str_replace_all("(\\r\\n\\s{3,}|\\r\\n\\t|\\s{2}\\r\\n\\s{2})", " ") %>%

    # 6) Double-space cleanup
    str_replace_all(" {2,}", " ") %>%

    # 7) Inject empty timestamps if speaker tags appear "bare"
    str_replace_all(paste0("(?<!\\r\\n)<v ", SPK, ">"),
                    "\r\n\r\n00:00:00.000 --> 00:00:00.000\r\n<v \\1>") %>%
    str_replace_all(paste0("(?<!000\\r\\n)<v ", SPK, ">"),
                    "\r\n00:00:00.000 --> 00:00:00.000\r\n<v \\1>") %>%

    # 8) Extra collapsing when consecutive speaker codes repeat
```

```
    str_replace_all(
      paste0("(?<=<v PAR>)(.*)\\r\\n\\r\\n\\r?\\n?(", TS, ") --> (", TS, ")\\r\\n<v PAR>"),
      "\\1 "
    ) %>%
    str_replace_all(
      paste0("(?<=<v (INT|INV)>)(.*)\\r\\n\\r\\n\\r?\\n?(", TS, ") --> (", TS, ")\\r\\n<v (INT|INV)>"),
      "\\2 "
    ) %>%

    # 9) Collapse triple blank lines
    str_replace_all("\\r\\n\\r\\n\\r\\n", "\r\n\r\n")
}
```

```
files$Text <- vapply(files$Text, transcribe_2, FUN.VALUE = character(1))
```

Step 3 – Timestamp reconstruction and reassignment:

This stage reconstructs and standardizes timestamp intervals across the entire transcript to ensure that temporal segmentation is internally consistent and computationally valid.

```
assign_timestamps <- function(name, txt, endTime) {

  cat("Assigning new timestamps to ", name, "...\n", sep = "")

  # 1) Extract start times as character
  starts_raw <- str_extract_all(txt, "(\\d{1,2}:\\d{2}:\\d{2}\\.\\d{3}) -->", simplify = TRUE)
  starts_raw <- starts_raw[starts_raw != ""]
  ts <- data.frame(start = str_replace_all(starts_raw, " -->", ""), stringsAsFactors = FALSE)

  # 1.1) Pre-allocate end column
  ts$end <- NA_character_

  # 1.2) End time = next start time; last end = provided endTime
  if (nrow(ts) > 1) ts$end[1:(nrow(ts) - 1)] <- ts$start[2:nrow(ts)]
  ts$end[nrow(ts)] <- endTime

  # 2) Attach dialogue text segments
  split <- t(str_split(
    txt,
    "(\\d{1,2}:\\d{2}:\\d{2}\\.\\d{3}) --> (\\d{1,2}:\\d{2}:\\d{2}\\.\\d{3})",
    simplify = TRUE
  ))
  split <- split[-1, ]

  if (nrow(ts) == length(split)) {
    ts$text <- split
  } else {
    cat("ERROR! There is likely a major formatting issue in the following transcript:", name, "\n")
    # Keep going with best effort (matches original "print error but continue" behavior)
    ts$text <- split
  }
```

```r
# 3) Remove milliseconds temporarily and convert to chron
ts$start <- str_replace_all(ts$start, "(\\d{1,2}:\\d{2}:\\d{2})\\.\\d{3}", "\\1")
ts$end   <- str_replace_all(ts$end,   "(\\d{1,2}:\\d{2}:\\d{2})\\.\\d{3}", "\\1")

ts$start <- chron(times = ts$start)
ts$end   <- chron(times = ts$end)

# 4) Interpolate missing (00:00:00) end times
zero_t <- chron(times = "00:00:00")

for (i in seq_len(nrow(ts))) {
  non_zero <- which(ts$end != zero_t)

  cond1 <- (ts$end[i] == zero_t && ts$start[i] != zero_t)
  cond2 <- (ts$end[i] == zero_t && ts$start[i] == zero_t && i == 1)

  if (cond1 || cond2) {
    next_candidates <- non_zero[non_zero > i]
    if (length(next_candidates) == 0) next

    next_r <- next_candidates[1]
    next_t <- ts$end[next_r]

    diff <- next_r - i
    increment <- (next_t - ts$start[i]) / (diff + 1)

    # 4.1) Debug column without changing outcomes
    ts[i, 4] <- paste(next_r, diff, increment)

    for (j in seq_len(diff)) {
      if (j == 1) ts$end[i] <- ts$start[i] + (increment * j)
      ts$start[i + j] <- ts$start[i] + (increment * j)
      ts$end[i + j]   <- ts$start[i] + (increment * (j + 1))
    }
  }
}

# 5) Re-add .000 milliseconds
ts$new_start <- paste0(ts$start, ".000")
ts$new_end   <- paste0(ts$end,   ".000")

# 6) Avoid zero-length intervals
for (i in seq_len(nrow(ts))) {
  if (ts$start[i] == ts$end[i]) {
    ts$new_end[i] <- paste0(ts$end[i], ".250")
    if (i + 1 <= nrow(ts)) ts$new_start[i + 1] <- paste0(ts$start[i + 1], ".750")
  }
}

# 7) Rebuild transcript
ts$new_text <- paste0(ts$new_start, " --> ", ts$new_end, ts$text)
new_file <- paste(ts$new_text, collapse = "")
paste0("WEBVTT\r\n\r\n", new_file)
```

```
}
```

```
files$Text <- mapply(assign_timestamps, files$Name, files$Text, files$end_time, USE.NAMES = FALSE)
```

```
## Assigning new timestamps to Cleaned_COBIVA122_Transcript_Round1.txt...
```

Step 4 – Export Cleaned Transcripts

This final stage writes the processed transcripts to disk. Instead of changing the working directory, now its explicitly define an output directory and construct full file paths when saving.

```
# Define output directory explicitly
output_dir <- file.path(your_wd, "Post R Cleanup")

# Create directory if it does not exist
dir.create(output_dir, showWarnings = FALSE, recursive = TRUE)

# Write each cleaned transcript
for (i in seq_len(nrow(files))) {
  write_file(
    files$Text[i],
    file.path(output_dir, files$Name[i])
  )
}

cat("Cleanup script complete!\n")
```

```
## Cleanup script complete!
```